SGDD

Sistema de Gestion de Dulcerias

DOCUMENTO DE DISENO DE SOFTWARE (SDD)

Version 1.0.0 | 2026

Autor: Bryan Stiven Bonilla Rosas

Plataforma SaaS Multi-Tenant | Clean Architecture + DDD

INFORMACION DEL DOCUMENTO

Campo	Detalle
Titulo	Documento de Diseno de Software (SDD)
Sistema	SGDD - Sistema de Gestion de Dulcerias
Version	1.0.0
Autor	Bryan Stiven Bonilla Rosas
Fecha	2026
Estado	Activo
Arquitectura base	Clean Architecture + Domain-Driven Design (DDD)
Stack principal	FastAPI (Python) + React (TypeScript) + PostgreSQL + Redis
Estandar	IEEE 1016 - Standard for Software Design Descriptions

Historial de Versiones

Version	Fecha	Autor	Cambio
1.0.0	2026	Bryan Stiven Bonilla Rosas	Creacion inicial del SDD

1. INTRODUCCION

El presente Documento de Diseno de Software (SDD) describe la implementacion interna del sistema SGDD, detallando el diseno de sus APIs, modelos de datos, flujos de negocio, servicios, estrategia de errores y logs. El SDD actua como puente entre los requisitos del SRS y la implementacion real del codigo.

1.1 Proposito

- Describir el diseno detallado de cada modulo y servicio del sistema.
- Documentar todos los endpoints de la API REST con sus contratos de entrada y salida.
- Definir el modelo de datos con sus atributos, tipos y relaciones.
- Especificar los flujos de negocio criticos con diagramas de secuencia.
- Establecer la estrategia de manejo de errores y logging estructurado.

1.2 Stack tecnologico

Capa	Tecnologia	Version	Rol
Backend API	FastAPI	>=0.104	Framework principal REST, async/await, OpenAPI automatico
ORM	SQLAlchemy	>=2.0	Mapeo objeto-relacional, queries async, migraciones Alembic
Validacion	Pydantic	v2	Schemas de entrada/salida, validacion de tipos y formatos
Base de datos	PostgreSQL	15	Persistencia relacional con soporte JSON y full-text search
Cache	Redis	7	Cache de consultas, sesiones y rate limiting
Autenticacion	PyJWT + bcrypt	Latest	JWT RS256, hash de contrasenas con salt
Frontend	React + TypeScript	18 / 5	SPA con hooks, context API y React Query
HTTP Client	Axios	>=1.6	Cliente HTTP con interceptores para JWT y errores
ML / Prediccion	Prophet + sklearn	Latest	Modelos de serie temporal y clasificacion
NLP / Bot	spaCy	>=3.7	Extraccion de entidades y clasificacion de intenciones
Migraciones DB	Alembic	>=1.13	Control de versiones del esquema de la base de datos
Contenedores	Docker Compose	v2	Orquestacion de servicios en desarrollo y produccion

2. ARQUITECTURA DE SERVICIOS

El backend esta organizado en modulos verticales, cada uno responsable de un dominio de negocio. Cada modulo sigue la misma estructura de capas (Clean Architecture) de forma independiente.

2.1 Estructura de directorios del proyecto

```
sgdd/   app/     domain/                  # Entidades y reglas de negocio puras
entities/              # Producto, Venta, Cliente, Inventario...
value_objects/         # Dinero, Cantidad, TenantId...       repositories/
# Interfaces abstractas de repositorios       exceptions/            #
Excepciones de dominio     application/             # Casos de uso
(orquestacion)       ventas/         registrar_venta.py
consultar_ventas.py       inventario/       clientes/       reportes/
prediccion/       bot/     infrastructure/          # Implementaciones
concretas       db/                     # Repositorios SQLAlchemy, modelos ORM
cache/                 # Adaptador Redis       ml/                    # Carga y
ejecucion de modelos ML       nlp/                   # Pipeline spaCy para el
bot       auth/                   # JWT provider, RBAC middleware     api/
# Capa de presentacion (FastAPI)       v1/         routers/             # auth,
ventas, inventario, reportes...         schemas/             # Pydantic schemas
de entrada/salida         dependencies/        # Inyeccion de dependencias
FastAPI     core/       config.py              # Settings con pydantic-settings
logging.py             # Configuracion de logs estructurados
exceptions.py          # Handler global de excepciones
```

2.2 Modulos del sistema

Modulo	Ruta base API	Responsabilidad
Autenticacion	/api/v1/auth	Login, refresh token, logout, gestion de sesiones JWT
Ventas (POS)	/api/v1/ventas	Registro de ventas, consulta, cancelacion y recibos
Inventario	/api/v1/inventario	Stock actual, alertas, entradas de lotes, ajustes
Clientes	/api/v1/clientes	CRUD clientes, historial de compras, segmentacion
Reportes	/api/v1/reportes	KPIs, ventas por periodo/producto, exportacion Excel
Prediccion	/api/v1/predicciones	Forecast de demanda, entrenamiento de modelos ML
Bot	/api/v1/bot	Consultas en lenguaje natural, respuestas estructuradas
Usuarios	/api/v1/usuarios	CRUD usuarios, asignacion de roles, gestion de permisos
Tenants	/api/v1/tenants	Alta de tenants, configuracion, limites del plan

3. DISENO DE APIs

Todos los endpoints requieren header: Authorization: Bearer <JWT> | Content-Type: application/json | X-Tenant-ID se extrae automaticamente del JWT.

3.1 Modulo de Autenticacion

Metodo	Endpoint	Descripcion	Auth requerida
POST	/api/v1/auth/login	Autenticar usuario, retorna access_token + refresh_token	No
POST	/api/v1/auth/refresh	Renovar access token usando refresh token vigente	Refresh token
POST	/api/v1/auth/logout	Revocar refresh token del usuario	Si
GET	/api/v1/auth/me	Datos del usuario autenticado y su rol	Si

```
// POST /api/v1/auth/login // Request body: {   "email": "admin@dulceria.com",
"password": "SecurePass123!" }  // Response 200 OK: {   "access_token":
"eyJhbGciOiJSUzI1NiJ9...",   "refresh_token": "eyJhbGciOiJSUzI1NiJ9...",
"token_type": "bearer",   "expires_in": 3600,   "user": { "id": "uuid",
"nombre": "Ana Garcia", "rol": "administrador" } }
```

3.2 Modulo de Ventas (POS)

Metodo	Endpoint	Descripcion	Roles
POST	/api/v1/ventas	Registrar nueva venta con multiples productos	Vendedor, Admin
GET	/api/v1/ventas	Listar ventas con filtros (fecha, vendedor, estado)	Supervisor, Admin
GET	/api/v1/ventas/{id}	Detalle completo de una venta y sus productos	Todos
POST	/api/v1/ventas/{id}/cancelar	Cancelar venta y revertir stock (solo mismo dia)	Supervisor, Admin
GET	/api/v1/ventas/{id}/recibo	Generar recibo en PDF de la venta	Todos

```
// POST /api/v1/ventas // Request body: {   "cliente_id": "uuid-cliente",
// Opcional   "metodo_pago": "efectivo",            // efectivo | tarjeta |
transferencia   "items": [     { "producto_id": "uuid-1", "cantidad": 3,
"precio_unitario": 2500 },     { "producto_id": "uuid-2", "cantidad": 1,
"precio_unitario": 8000 }   ],   "descuento_porcentaje": 5,              //
Opcional, 0-100   "notas": "Cliente frecuente"           // Opcional }  //
Response 201 Created: {   "id": "uuid-venta",   "numero_recibo": "VTA-2026-
00142",   "total": 15125,   "subtotal": 15500,   "descuento": 375,   "fecha":
"2026-03-09T14:30:00Z",   "estado": "completada" }
```

3.3 Modulo de Inventario

Metodo	Endpoint	Descripcion	Roles
GET	/api/v1/inventario	Stock actual de todos los productos con alertas	Todos
GET	/api/v1/inventario/{producto_id}	Stock detallado de un producto especifico	Todos
POST	/api/v1/inventario/lotes	Registrar entrada de mercancia (lote)	Operador, Admin
GET	/api/v1/inventario/lotes	Historial de entradas de lotes	Supervisor, Admin
GET	/api/v1/inventario/alertas	Productos con stock por debajo del minimo	Todos
PUT	/api/v1/inventario/{producto_id}/ajuste	Ajuste manual de inventario con justificacion	Admin

3.4 Modulo de Reportes

Metodo	Endpoint	Descripcion
GET	/api/v1/reportes/ventas	Ventas por rango de fechas, producto, vendedor o metodo de pago
GET	/api/v1/reportes/inventario	Estado actual del inventario con valoracion de stock
GET	/api/v1/reportes/clientes	Ranking de clientes por frecuencia y volumen de compra
GET	/api/v1/reportes/kpis	KPIs del negocio: ventas del dia, semana, mes y comparativas
GET	/api/v1/reportes/exportar	Exportar reporte en formato Excel o CSV

3.5 Modulo de Prediccion y Bot

Metodo	Endpoint	Descripcion
GET	/api/v1/predicciones	Forecast de demanda para todos los productos (30 dias)
GET	/api/v1/predicciones/{producto_id}	Forecast detallado de un producto con intervalos de confianza
POST	/api/v1/predicciones/entrenar	Disparar reentrenamiento del modelo ML (Admin only)

POST	/api/v1/bot/consultar	Enviar consulta en lenguaje natural, recibir respuesta
GET	/api/v1/bot/historial	Historial de conversaciones del usuario autenticado

4. MODELO DE DATOS

El sistema utiliza PostgreSQL como base de datos relacional. Todas las tablas incluyen tenant_id como campo obligatorio para garantizar el aislamiento multi-tenant. Los IDs son UUID v4 generados en la aplicacion.

4.1 Tabla: tenants

Campo	Tipo	Restricciones	Descripcion
id	UUID	PK, NOT NULL	Identificador unico del tenant
nombre	VARCHAR(100)	NOT NULL	Nombre comercial de la dulceria
slug	VARCHAR(60)	UNIQUE, NOT NULL	Identificador URL-friendly
plan	VARCHAR(20)	NOT NULL, DEFAULT 'basico'	Plan de suscripcion: basico, pro, enterprise
activo	BOOLEAN	NOT NULL, DEFAULT true	Estado de la suscripcion
config	JSONB	NOT NULL, DEFAULT '{}'	Configuracion del tenant (moneda, zona horaria)
creado_en	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Fecha de creacion

4.2 Tabla: usuarios

Campo	Tipo	Restricciones	Descripcion
id	UUID	PK, NOT NULL	Identificador del usuario
tenant_id	UUID	FK tenants.id, NOT NULL	Tenant al que pertenece
nombre	VARCHAR(100)	NOT NULL	Nombre completo
email	VARCHAR(255)	UNIQUE por tenant, NOT NULL	Correo electronico
password_hash	VARCHAR(255)	NOT NULL	Hash bcrypt de la contrasena
rol	VARCHAR(20)	NOT NULL	administrador | supervisor | vendedor | operador
activo	BOOLEAN	DEFAULT true	Estado del usuario
ultimo_login	TIMESTAMPTZ	NULLABLE	Fecha del ultimo ingreso al sistema
creado_en	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Fecha de creacion

4.3 Tabla: productos

Campo	Tipo	Restricciones	Descripcion
id	UUID	PK, NOT NULL	Identificador del producto
tenant_id	UUID	FK tenants.id, NOT NULL	Tenant propietario
nombre	VARCHAR(150)	NOT NULL	Nombre del producto
descripcion	TEXT	NULLABLE	Descripcion detallada
categoria	VARCHAR(80)	NOT NULL	Categoria del producto
precio	NUMERIC(12,2)	NOT NULL, CHECK > 0	Precio de venta en COP
costo	NUMERIC(12,2)	NULLABLE	Costo de compra
unidad_medida	VARCHAR(20)	NOT NULL	unidad | caja | kg | litro
stock_minimo	INTEGER	NOT NULL, DEFAULT 5	Nivel minimo para disparar alerta
activo	BOOLEAN	DEFAULT true	Estado del producto
creado_en	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Fecha de creacion

4.4 Tabla: inventario

Campo	Tipo	Restricciones	Descripcion
id	UUID	PK, NOT NULL	Identificador del registro
tenant_id	UUID	FK tenants.id, NOT NULL	Tenant propietario
producto_id	UUID	FK productos.id, NOT NULL	Producto al que pertenece
cantidad	INTEGER	NOT NULL, CHECK >= 0	Stock actual disponible
actualizado_en	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Ultima actualizacion del stock

4.5 Tabla: lotes_inventario

Campo	Tipo	Restricciones	Descripcion
id	UUID	PK, NOT NULL	Identificador del lote

tenant_id	UUID	FK tenants.id, NOT NULL	Tenant propietario
producto_id	UUID	FK productos.id, NOT NULL	Producto recibido
cantidad	INTEGER	NOT NULL, CHECK > 0	Cantidad recibida
costo_unitario	NUMERIC(12,2)	NULLABLE	Costo unitario del lote
proveedor	VARCHAR(150)	NULLABLE	Nombre del proveedor
fecha_vencimiento	DATE	NULLABLE	Fecha de vencimiento del lote
recibido_por	UUID	FK usuarios.id	Usuario que registro la entrada
notas	TEXT	NULLABLE	Observaciones del lote
creado_en	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Fecha de registro

4.6 Tabla: ventas

Campo	Tipo	Restricciones	Descripcion
id	UUID	PK, NOT NULL	Identificador de la venta
tenant_id	UUID	FK tenants.id, NOT NULL	Tenant propietario
numero_recibo	VARCHAR(30)	UNIQUE por tenant	Numero secuencial: VTA-2026-00142
vendedor_id	UUID	FK usuarios.id, NOT NULL	Usuario que realizo la venta
cliente_id	UUID	FK clientes.id, NULLABLE	Cliente (opcional)
subtotal	NUMERIC(14,2)	NOT NULL	Suma de items sin descuento
descuento	NUMERIC(14,2)	NOT NULL, DEFAULT 0	Monto descontado
total	NUMERIC(14,2)	NOT NULL	Monto final cobrado
metodo_pago	VARCHAR(20)	NOT NULL	efectivo | tarjeta | transferencia
estado	VARCHAR(20)	NOT NULL, DEFAULT 'completada'	completada | cancelada
notas	TEXT	NULLABLE	Observaciones de la venta
creado_en	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Fecha y hora de la venta

4.7 Tabla: detalle_ventas

Campo	Tipo	Restricciones	Descripcion
id	UUID	PK, NOT NULL	Identificador de la linea
venta_id	UUID	FK ventas.id, NOT NULL	Venta a la que pertenece
producto_id	UUID	FK productos.id, NOT NULL	Producto vendido
cantidad	INTEGER	NOT NULL, CHECK > 0	Cantidad vendida
precio_unitario	NUMERIC(12,2)	NOT NULL	Precio en el momento de la venta
subtotal	NUMERIC(14,2)	NOT NULL	cantidad * precio_unitario

4.8 Indices de la base de datos

Tabla	Indice	Tipo	Proposito
ventas	idx_ventas_tenant_fecha	BTREE	Consultas de reportes por rango de fechas
ventas	idx_ventas_vendedor	BTREE	Ventas por vendedor
productos	idx_productos_tenant_cat	BTREE	Filtros por categoria
inventario	idx_inv_tenant_producto	BTREE UNIQUE	Constraint: un registro por producto/tenant
usuarios	idx_users_email_tenant	BTREE UNIQUE	Email unico por tenant
detalle_ventas	idx_detalle_venta_id	BTREE	Join con tabla ventas
lotes_inv	idx_lotes_vencimiento	BTREE	Consulta de lotes proximos a vencer

5. FLUJOS DE NEGOCIO

5.1 Flujo: Registrar Venta (POS)

Es el flujo critico del sistema. Involucra validacion de stock, persistencia atomica y descuento de inventario dentro de una transaccion de base de datos.

```
Vendedor          API FastAPI        RegistrarVentaUseCase    DB (PostgreSQL)
Redis     |                   |                      |                    |
|      |-- POST /ventas --> |                      |                    |
|      |                    |-- Validar JWT ------> |                    |
|      |                    |-- Validar schema ---> |                    |
|      |                    |                       |-- BEGIN TX ------> |
|      |                    |                       |-- Leer stock ----> |
|      |                    |                       |<-- stock actual -- |
|      |                    |                       |-- Validar stock    |
|      |                    |                       |   (si insuf. RAISE)|
|      |                    |                       |-- INSERT venta --> |
|      |                    |                       |-- INSERT detalles->|
|      |                    |                       |-- UPDATE stock --> |
|      |                    |                       |-- COMMIT TX -----> |
|      |                    |                       |                    |--
Invalidar ->|     |                   |                      |
|   cache stock |     |<-- 201 + recibo - |                      |
|               |
```

5.2 Flujo: Consulta al Bot Conversacional

```
Usuario           API FastAPI       ConsultarBotUseCase      NLP Pipeline
DB     |                  |                    |                     |
|      |-- POST /bot  --> |                    |                     |
|      |   { texto }      |-- Validar JWT ---> |                     |
|      |                  |                    |-- tokenizar() ----> |
|      |                  |                    |                     |-- spaCy
NER  |     |                  |                    |<-- intencion+ents - |
|      |                  |                    |-- build_query()     |
|      |                  |                    |   (SQL dinamico)    |
|      |                  |                    |-- ejecutar query --------------
---> |     |                  |                    |<-- resultado -------------
--------- |     |                  |                    |--
formatear_respuesta()             |     |<-- 200 + resp -- |
|                                 |
```

5.3 Flujo: Prediccion de Demanda

```
  [Scheduler semanal]        |   ProcesarPrediccionUseCase        |        |--
Cargar historial de ventas (12 meses) desde PostgreSQL        |-- Preprocesar
datos (agrupar por dia, rellenar huecos)        |-- Entrenar modelo Prophet por
producto        |-- Generar forecast (7, 14 y 30 dias)        |-- Calcular
intervalos de confianza al 95%        |-- Evaluar MAE y RMSE del modelo
|-- Persistir resultados en tabla predicciones        |-- Invalidar cache de
predicciones en Redis        |-- Registrar metricas en Prometheus
```

5.4 Flujo: Autenticacion y autorizacion

```
  Request HTTP        |   [Middleware JWT]        |-- Extraer header
'Authorization: Bearer <token>'        |-- Verificar firma RS256 del JWT
|-- Verificar expiracion (exp claim)        |-- Extraer tenant_id, user_id, rol
del payload        |-- Inyectar CurrentUser en el contexto de la request
|   [Middleware RBAC]        |-- Verificar que el rol tiene permiso para el
endpoint        |-- Si no: raise HTTP 403 Forbidden        |   [Middleware
Multi-Tenant]        |-- Inyectar tenant_id en todos los queries del ORM
|-- Filtro automatico: WHERE tenant_id = :tenant_id
```

6. DISENO DE CASOS DE USO

Cada caso de uso es una clase Python que recibe sus dependencias por inyeccion (repositorios, servicios). Esto garantiza que son testeables de forma aislada sin necesidad de base de datos real.

6.1 RegistrarVentaUseCase

```
class RegistrarVentaUseCase:     def __init__(         self,
venta_repo: AbstractVentaRepository,         inventario_repo:
AbstractInventarioRepository,         cliente_repo: AbstractClienteRepository,
cache: AbstractCacheService     ): ...      async def ejecutar(self, cmd:
RegistrarVentaCommand) -> VentaResult:         # 1. Validar que todos los
productos existen en el tenant         # 2. Validar stock suficiente para cada
item         # 3. Calcular subtotal, descuento y total         # 4. Crear
entidad Venta en el dominio         # 5. Persistir venta + detalles en TX
atomica         # 6. Decrementar stock en inventario         # 7. Invalidar
cache de inventario         # 8. Retornar VentaResult con recibo
```

6.2 Patron Command + Result

```
# Command: datos de entrada validados @dataclass class RegistrarVentaCommand:
tenant_id: UUID     vendedor_id: UUID     cliente_id: Optional[UUID]     items:
List[ItemVentaDTO]     metodo_pago: MetodoPago     descuento_porcentaje:
Decimal = Decimal('0')  # Result: datos de salida tipados @dataclass class
VentaResult:     id: UUID     numero_recibo: str     total: Decimal     estado:
EstadoVenta     fecha: datetime
```

6.3 Diseno del Repositorio (interfaz abstracta)

```
# Interfaz en el dominio (sin dependencia de SQLAlchemy) class
AbstractVentaRepository(ABC):     @abstractmethod     async def guardar(self,
venta: Venta) -> Venta: ...      @abstractmethod     async def
obtener_por_id(self, id: UUID, tenant_id: UUID) -> Optional[Venta]: ...
@abstractmethod     async def listar(self, tenant_id: UUID, filtros:
FiltrosVenta) -> Page[Venta]: ...  # Implementacion en infraestructura (usa
SQLAlchemy) class SQLAlchemyVentaRepository(AbstractVentaRepository):     def
__init__(self, session: AsyncSession): ...      async def guardar(self, venta:
Venta) -> Venta:         orm_model = VentaORM.from_domain(venta)
self.session.add(orm_model)         await self.session.flush()         return
orm_model.to_domain()
```

7. DISENO DE WORKERS Y TAREAS ASINCRONAS

El sistema utiliza workers para tareas que no deben bloquear el flujo de la API: reentrenamiento de modelos ML, envio de alertas y generacion de reportes pesados.

Worker	Trigger	Frecuencia	Descripcion
PrediccionWorker	Scheduler (APScheduler)	Domingos 02:00 AM	Reentrenar modelos ML con datos de la ultima semana
AlertaStockWorker	Evento (post-venta)	Inmediato	Evaluar si algun producto llego al stock minimo
ReporteWorker	Request API asincrona	On-demand	Generar reportes Excel pesados en background
AuditWorker	Hook (post-operacion)	Inmediato	Persistir log de auditoria de operaciones criticas
LimpiezaWorker	Scheduler diario	03:00 AM	Purgar logs y sesiones expiradas de Redis

```
# PrediccionWorker - ejecucion semanal con APScheduler from
apscheduler.schedulers.asyncio import AsyncIOScheduler  scheduler =
AsyncIOScheduler()  @scheduler.scheduled_job('cron', day_of_week='sun', hour=2)
async def ejecutar_prediccion_semanal():     tenants_activos = await
tenant_repo.listar_activos()     for tenant in tenants_activos:         await
prediccion_use_case.ejecutar(tenant_id=tenant.id)
logger.info('prediccion_completada', tenant_id=str(tenant.id))
```

8. INTEGRACIONES EXTERNAS

Integracion	Tipo	Descripcion	Estado
Redis	Cache / Rate Limit	Cache de consultas frecuentes y control de rate limiting por IP	Implementado
PostgreSQL	Base de datos	Persistencia principal con soporte async via asyncpg	Implementado
Prometheus	Metricas	Exposicion de metricas de la API en /metrics	Implementado
Grafana	Observabilidad	Dashboards de latencia, errores y uso del sistema	Implementado
SMTP / Email	Notificaciones	Envio de alertas de stock y reportes programados	Planificado
S3 / Storage	Archivos	Almacenamiento de reportes Excel y archivos adjuntos	Planificado

8.1 Estrategia de cache con Redis

```
# Patron Cache-Aside implementado en el servicio async def
obtener_inventario(tenant_id: UUID, producto_id: UUID):     cache_key =
f"inventario:{tenant_id}:{producto_id}"      # 1. Intentar desde cache
cached = await redis.get(cache_key)     if cached:         return
InventarioDTO.parse_raw(cached)      # 2. Consultar DB si cache miss
inventario = await inventario_repo.obtener(tenant_id, producto_id)      # 3.
Guardar en cache con TTL de 5 minutos     await redis.setex(cache_key, 300,
inventario.json())     return inventario  # Invalidacion al actualizar stock:
async def invalidar_cache_inventario(tenant_id: UUID, producto_id: UUID):
await redis.delete(f"inventario:{tenant_id}:{producto_id}")
```

9. GESTION DE ERRORES

El sistema implementa un handler global de excepciones que transforma errores del dominio en respuestas HTTP estandarizadas. El cliente siempre recibe una respuesta JSON consistente.

9.1 Codigos de error HTTP

Codigo HTTP	Tipo de error	Descripcion
400 Bad Request	Validacion de schema	El body o los query params no cumplen el schema Pydantic
401 Unauthorized	Autenticacion fallida	JWT invalido, expirado o ausente
403 Forbidden	Autorizacion denegada	El rol del usuario no tiene permiso para el recurso
404 Not Found	Recurso no encontrado	El ID solicitado no existe o no pertenece al tenant
409 Conflict	Conflicto de negocio	Ej: email ya registrado, numero de recibo duplicado
422 Unprocessable Entity	Error de dominio	Ej: stock insuficiente, fecha de vencimiento invalida
429 Too Many Requests	Rate limit excedido	Mas de 100 requests/min por IP
500 Internal Server Error	Error no controlado	Error inesperado, se registra en logs y Prometheus

9.2 Formato estandar de respuesta de error

```
// Todas las respuestas de error siguen este formato: {   "error": {
"code": "STOCK_INSUFICIENTE",          // Codigo interno del error
"message": "Stock insuficiente para el producto solicitado",     "details": {
// Contexto adicional (opcional)       "producto_id": "uuid-123",
"stock_disponible": 2,       "cantidad_solicitada": 5     },     "timestamp":
"2026-03-09T14:30:00Z",     "request_id": "req-abc-123"            // Para
correlacion de logs   } }
```

9.3 Excepciones del dominio

Excepcion	Codigo HTTP	Codigo de error
StockInsuficienteError	422	STOCK_INSUFICIENTE
ProductoNoEncontradoError	404	PRODUCTO_NO_ENCONTRADO
TenantNoAutorizadoError	403	ACCESO_NO_AUTORIZADO
VentaCanceladaError	409	VENTA_YA_CANCELADA

LoteExpiradoError	422	LOTE_FECHA_INVALIDA
CredencialesInvalidasError	401	CREDENCIALES_INVALIDAS

10. ESTRATEGIA DE LOGS Y OBSERVABILIDAD

El sistema utiliza logs estructurados en formato JSON para facilitar la ingesta en herramientas de observabilidad. Cada log incluye campos de correlacion que permiten rastrear una request a traves de todos los servicios.

10.1 Campos estandar en cada log

Campo	Tipo	Descripcion
timestamp	ISO 8601	Fecha y hora exacta del evento en UTC
level	string	DEBUG | INFO | WARNING | ERROR | CRITICAL
event	string	Nombre del evento: 'venta_registrada', 'stock_alerta', etc.
tenant_id	UUID	Tenant que origino el evento
user_id	UUID	Usuario autenticado que realizo la accion
request_id	UUID	ID unico de la request HTTP para correlacion
module	string	Modulo de la aplicacion que emite el log
duration_ms	integer	Duracion de la operacion en milisegundos
extra	JSONB	Contexto adicional especifico del evento

10.2 Eventos criticos que se loguean

Evento	Nivel	Descripcion
login_exitoso	INFO	Usuario autenticado correctamente
login_fallido	WARNING	Intento de login con credenciales invalidas
venta_registrada	INFO	Venta completada con monto y numero de recibo
stock_alerta	WARNING	Producto alcanzo el nivel minimo de stock
stock_agotado	ERROR	Producto con stock en 0
acceso_denegado	WARNING	Request bloqueada por RBAC o multi-tenant
prediccion_completada	INFO	Modelo ML entrenado y forecast generado
error_no_controlado	CRITICAL	Excepcion inesperada capturada por el handler global
rate_limit_excedido	WARNING	IP bloqueada por exceder el limite de requests

```
// Ejemplo de log estructurado en JSON: {   "timestamp": "2026-03-
09T14:30:01.234Z",   "level": "INFO",   "event": "venta_registrada",
"tenant_id": "uuid-tenant-abc",   "user_id": "uuid-vendedor-xyz",
"request_id": "req-9f8a7b6c",   "module": "application.ventas",
"duration_ms": 87,   "extra": {     "venta_id": "uuid-venta-001",
```

```
"numero_recibo": "VTA-2026-00142",     "total": 15125,     "items_count": 3   }
}
```


Estado documental del proyecto SGDD

Documento	Estado
SGDD - Investigacion y contexto del negocio	Completo
SRS - Especificacion de Requisitos de Software	Completo
SAD - Documento de Arquitectura de Software	Completo
SDD - Documento de Diseno de Software (este doc)	Completo
STP - Plan Maestro de Pruebas	Completo
ADR - Architecture Decision Records	Completo

Documentacion tecnica completa alineada a: IEEE 1016 | ISO 29148 | ISO 25010 | ISO 29119 | ISO 12207 | ISO 27001